

Computer Vision from **Pixels** to Production Models

A structured, hands-on reference covering image processing fundamentals through deep learning-based detection, tracking, and deployment — with a practice task for every topic.

27

Topics Covered

2

Skill Levels

27

Practice Tasks

CONTENTS

What's Inside

PART I — Image Processing Foundations

1.1	What Is Computer Vision?	4
1.2	Color Spaces & Channels	4
1.3	Image Resizing & Cropping	5
1.4	Image Filtering & Blurring	5
1.5	Edge Detection	5
1.6	Thresholding & Binary Masks	6
1.7	Contours & Shape Detection	6
1.8	Drawing Shapes & Annotations	7
1.9	Reading Video & Webcam Streams	7
1.10	Image Datasets & Annotation Formats	7
1.11	Image Augmentation Basics	8
1.12	Intro to Image Classification	8

PART II — Deep Learning & Deployment

2.1	Convolutional Neural Networks (CNNs)	10
2.2	Transfer Learning with Pretrained Backbones	10
2.3	Object Detection with YOLO	11
2.4	Non-Max Suppression (NMS)	11
2.5	Semantic & Instance Segmentation	11
2.6	Object Tracking Across Frames	12
2.7	Custom Dataset Training Pipeline	12
2.8	Model Training & Fine-Tuning	13
2.9	Evaluation Metrics: IoU, Precision, Recall, mAP	13
2.10	Pose Estimation & Keypoint Detection	14
2.11	OCR: Reading Text from Images	14
2.12	Working with RTSP & IP Camera Streams	15
2.13	Model Optimization: Quantization & ONNX Export	15
2.14	Building a Real-Time Inference Pipeline	16
2.15	Deploying CV Models with FastAPI	16

Appendix

A	Quick Reference Cheat Sheets	16
B	Practice Checklist & Next Steps	18

PART I

Image Processing Foundations

Start here if you're new to computer vision. These twelve topics cover the essential building blocks — pixels, color spaces, filtering, contours, and basic classification — that every CV pipeline is built from.

Pixels & Color

Filtering

Edge Detection

Contours

Video I/O

Classification

1. Image Processing Foundations

Core OpenCV skills & classical CV — topics 1.1–1.12

1.1 What Is Computer Vision?

Computer vision teaches machines to interpret visual data — images and video — and extract meaningful information such as objects, motion, or text. A digital image is simply a grid of pixel intensity values that OpenCV loads as NumPy arrays.

```
LOAD_IMAGE.PY

import cv2

img = cv2.imread("photo.jpg")
print("Shape:", img.shape)  # (height, width, channels)
print("Dtype:", img.dtype)  # usually uint8

cv2.imshow("Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

TRY IT YOURSELF

Load any image with OpenCV and print its height, width, and number of color channels.

1.2 Color Spaces & Channels

Images are commonly stored in BGR (OpenCV's default), RGB, or grayscale. Converting between color spaces like HSV is essential for tasks such as color-based object detection.

```
COLOR_SPACES.PY

import cv2

img = cv2.imread("photo.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

cv2.imwrite("gray.jpg", gray)
cv2.imwrite("hsv.jpg", hsv)
```

TRY IT YOURSELF

Convert an image to HSV and print the Hue, Saturation, and Value of the pixel at position (0, 0).

1.3 Image Resizing & Cropping

Resizing controls the input dimensions expected by most models, while cropping isolates a region of interest (ROI) using simple array slicing.

```
RESIZE_CROP.PY

import cv2

img = cv2.imread("photo.jpg")
resized = cv2.resize(img, (416, 416))

# Crop: rows 50:200, columns 100:300
cropped = img[50:200, 100:300]

cv2.imwrite("resized.jpg", resized)
cv2.imwrite("cropped.jpg", cropped)
```

TRY IT YOURSELF

Resize an image to 50% of its original dimensions using `cv2.resize()` with `fx` and `fy` scale factors.

1.4 Image Filtering & Blurring

Blurring (Gaussian, median) reduces noise before edge detection, while sharpening kernels enhance detail. Filters use a sliding-window convolution over the image.

```
FILTERING.PY

import cv2

img = cv2.imread("photo.jpg")
blurred = cv2.GaussianBlur(img, (5, 5), 0)
median = cv2.medianBlur(img, 5)

cv2.imwrite("blurred.jpg", blurred)
cv2.imwrite("median.jpg", median)
```

TRY IT YOURSELF

Apply Gaussian blur with three different kernel sizes (3x3, 9x9, 15x15) and compare the results.

1.5 Edge Detection

The Canny edge detector finds boundaries by locating rapid intensity changes, forming the basis for shape and contour analysis.

```
EDGES.PY

import cv2

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
blurred = cv2.GaussianBlur(img, (5, 5), 0)
edges = cv2.Canny(blurred, threshold1=50, threshold2=150)

cv2.imwrite("edges.jpg", edges)
```

TRY IT YOURSELF

Run Canny edge detection with different threshold pairs and observe how edge density changes.

1.6 Thresholding & Binary Masks

Thresholding converts a grayscale image into a binary (black/white) mask, useful for isolating bright objects or preparing images for contour detection.

```
THRESHOLD.PY

import cv2

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
adaptive = cv2.adaptiveThreshold(
    img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 11, 2
)
cv2.imwrite("binary.jpg", binary)
```

TRY IT YOURSELF

Create a binary mask that isolates only the brightest 20% of pixels in a grayscale image.

1.7 Contours & Shape Detection

Contours are continuous curves joining boundary points of connected regions — useful for counting objects, measuring area, or approximating shapes.

```
CONTOURS.PY

import cv2

img = cv2.imread("shapes.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

contours, _ = cv2.findContours(
    thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)
print("Shapes found:", len(contours))
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)
```

TRY IT YOURSELF

Count how many separate objects appear in a thresholded image using `cv2.findContours()`.

1.8 Drawing Shapes & Annotations

OpenCV's drawing functions overlay rectangles, circles, lines, and text — the same primitives used to visualize bounding boxes and detection results.

```
DRAWING.PY

import cv2

img = cv2.imread("photo.jpg")
cv2.rectangle(img, (50, 50), (200, 200), (0, 255, 0), 2)
cv2.circle(img, (300, 300), 40, (255, 0, 0), -1)
cv2.putText(img, "Detected", (50, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
cv2.imwrite("annotated.jpg", img)
```

TRY IT YOURSELF

Draw a bounding box around any object in an image and label it with a confidence score like '0.92'.

1.9 Reading Video & Webcam Streams

cv2.VideoCapture reads frames from a video file, an RTSP/IP camera stream, or a connected webcam, one frame at a time in a loop.

```
VIDEO_STREAM.PY

import cv2

cap = cv2.VideoCapture(0) # 0 = default webcam

while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("Live Feed", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

TRY IT YOURSELF

Modify the loop to save every 30th frame from a video file as a separate JPG image.

1.10 Image Datasets & Annotation Formats

Supervised CV models learn from labeled datasets. Common annotation formats include COCO (JSON), Pascal VOC (XML), and YOLO (normalized text files) — each storing bounding boxes differently.

```
YOLO_LABEL.TXT

# class_id x_center y_center width height (all normalized 0-1)
0 0.512 0.487 0.230 0.340
2 0.125 0.650 0.080 0.120
```

TRY IT YOURSELF

Take one image and manually write its YOLO-format label line for a single bounding box you estimate by eye.

1.11 Image Augmentation Basics

Augmentation artificially expands training data through flips, rotations, brightness shifts, and noise — improving a model's ability to generalize.

```
AUGMENTATION.PY

import cv2
import numpy as np

img = cv2.imread("photo.jpg")
flipped = cv2.flip(img, 1) # horizontal flip

bright = cv2.convertScaleAbs(img, alpha=1.2, beta=30)

(h, w) = img.shape[:2]
matrix = cv2.getRotationMatrix2D((w/2, h/2), 15, 1.0)
rotated = cv2.warpAffine(img, matrix, (w, h))
```

TRY IT YOURSELF

Apply a horizontal flip and a 15-degree rotation to an image, saving both as new files.

1.12 Intro to Image Classification

Classification assigns a single label to an entire image. A simple pretrained model like MobileNet can classify everyday objects out of the box using transfer learning.

```
CLASSIFY.PY

import torch
from torchvision import models, transforms
from PIL import Image

model = models.mobilenet_v2(pretrained=True)
model.eval()

transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
])

img = transform(Image.open("photo.jpg")).unsqueeze(0)
output = model(img)
print("Predicted class:", output.argmax().item())
```

TRY IT YOURSELF

Run MobileNetV2 on three different images and print the predicted class index for each.

PART II

Deep Learning & Deployment

Once the fundamentals feel natural, these fifteen topics take you into CNNs, real-time object detection, tracking, segmentation, and the tooling used to train, evaluate, and deploy production CV models.

CNNs

YOLO Detection

Tracking

Segmentation

OCR

Deployment

2. Deep Learning & Deployment

CNNs, detection, tracking & production tooling — topics 2.1–2.15

2.1 Convolutional Neural Networks (CNNs)

CNNs use learnable filters (kernels) that slide across an image to detect patterns like edges, textures, and shapes, building up to higher-level features through stacked layers.

```
SIMPLE_CNN.PY

import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(32 * 112 * 112, num_classes)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

TRY IT YOURSELF

Draw out on paper (or in comments) how the tensor shape changes after each layer for a 224x224 RGB input.

2.2 Transfer Learning with Pretrained Backbones

Instead of training from scratch, transfer learning fine-tunes a model already trained on a large dataset (like ImageNet) — drastically reducing data and compute needs.

```
TRANSFER_LEARNING.PY

import torch.nn as nn
from torchvision import models

model = models.resnet18(pretrained=True)

# Freeze all layers except the final classifier
for param in model.parameters():
    param.requires_grad = False

model.fc = nn.Linear(model.fc.in_features, num_classes=5)
# Only model.fc will be trained now
```

TRY IT YOURSELF

Load a pretrained ResNet18, freeze its layers, and replace the final layer for a 3-class problem.

2.3 Object Detection with YOLO

YOLO (You Only Look Once) predicts bounding boxes and class labels in a single forward pass, making it fast enough for real-time detection — the same family of models used in traffic and safety-monitoring pipelines.

```
YOLO_INFERENCE.PY

from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model("photo.jpg")

for box in results[0].boxes:
    cls = int(box.cls[0])
    conf = float(box.conf[0])
    xxyy = box.xyxy[0].tolist()
    print(f"Class {cls}, conf {conf:.2f}, box {xxyy}")
```

TRY IT YOURSELF

Run YOLOv8 on an image containing multiple objects and print the count of detections per class.

2.4 Non-Max Suppression (NMS)

Detectors often produce multiple overlapping boxes for the same object. NMS keeps only the highest-confidence box among overlapping candidates, measured by Intersection over Union (IoU).

```
NMS.PY

import cv2

boxes = [[50, 50, 150, 150], [55, 55, 145, 145], [300, 300, 400, 400]]
scores = [0.95, 0.80, 0.90]

indices = cv2.dnn.NMSBoxes(boxes, scores, score_threshold=0.5, nms_threshold=0.4)
print("Kept boxes:", indices.flatten())
```

TRY IT YOURSELF

Compute IoU by hand for two overlapping boxes and verify it matches what NMS would filter out.

2.5 Semantic & Instance Segmentation

Segmentation assigns a class label to every pixel. Semantic segmentation labels pixels by category; instance segmentation additionally separates individual object instances.

```
SEGMENTATION.PY

from ultralytics import YOLO

model = YOLO("yolov8n-seg.pt")
results = model("street.jpg")

mask = results[0].masks.data[0].numpy()
print("Mask shape:", mask.shape)
print("Pixels covered:", mask.sum())
```

TRY IT YOURSELF

Run a segmentation model on an image and calculate what percentage of pixels belong to the largest detected object.

2.6 Object Tracking Across Frames

Tracking assigns a persistent ID to a detected object as it moves across video frames, using algorithms like SORT or DeepSORT that combine motion prediction with appearance matching.

```
TRACKING.PY

from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model.track(source="traffic.mp4", tracker="bytetrack.yaml")

for r in results:
    for box in r.boxes:
        track_id = int(box.id[0]) if box.id is not None else -1
        print("Tracking ID:", track_id)
```

TRY IT YOURSELF

Process a short video and count how many unique tracking IDs cross a fixed horizontal line in the frame.

2.7 Custom Dataset Training Pipeline

Training a custom detector means organizing images and labels into train/val splits, defining classes, and pointing a training config at that structure.

```
DATA.YAML

train: dataset/images/train
val: dataset/images/val

nc: 2
names: ['helmet', 'no_helmet']
```

TRY IT YOURSELF

Sketch out a folder structure (images/, labels/, data.yaml) for a 2-class custom detection dataset.

2.8 Model Training & Fine-Tuning

Fine-tuning trains a pretrained model on your own dataset for a limited number of epochs, monitoring loss and validation metrics like mAP (mean Average Precision).

```
TRAIN.PY

from ultralytics import YOLO

model = YOLO("yolo8n.pt")
model.train(
    data="data.yaml",
    epochs=50,
    imgsz=640,
    batch=16
)

metrics = model.val()
print("mAP50:", metrics.box.map50)
```

TRY IT YOURSELF

After training, note which epoch achieved the best validation mAP and why more epochs isn't always better.

2.9 Evaluation Metrics: IoU, Precision, Recall, mAP

IoU measures box overlap quality; precision and recall measure detection correctness and completeness; mAP averages precision across confidence thresholds and classes — the standard benchmark for detector quality.

```
METRICS.PY

def iou(boxA, boxB):
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])

    inter = max(0, xB - xA) * max(0, yB - yA)
    areaA = (boxA[2]-boxA[0]) * (boxA[3]-boxA[1])
    areaB = (boxB[2]-boxB[0]) * (boxB[3]-boxB[1])
    return inter / float(areaA + areaB - inter)
```

TRY IT YOURSELF

Implement the iou() function above and test it on two boxes that overlap by roughly half their area.

2.10 Pose Estimation & Keypoint Detection

Pose estimation locates specific keypoints on a subject (joints on a human body, facial landmarks), enabling applications like posture analysis or gesture recognition.

```
POSE.PY

from ultralytics import YOLO

model = YOLO("yolov8n-pose.pt")
results = model("person.jpg")

keypoints = results[0].keypoints.xy[0]
print("Number of keypoints:", len(keypoints))
print("Nose position:", keypoints[0])
```

TRY IT YOURSELF

Run pose estimation on an image of a person and print the coordinates of the shoulder keypoints.

2.11 OCR: Reading Text from Images

Optical Character Recognition extracts text from images — useful for reading license plates, screen displays, or scanned documents. Modern OCR pipelines combine text detection with recognition.

```
OCR_DEMO.PY

import easyocr

reader = easyocr.Reader(["en"])
results = reader.readtext("plate.jpg")

for (bbox, text, confidence) in results:
    print(f"Text: {text}, Confidence: {confidence:.2f}")
```

TRY IT YOURSELF

Run OCR on an image containing a license plate or a screen reading, and print only results with confidence above 0.7.

2.12 Working with RTSP & IP Camera Streams

IP cameras stream video over RTSP/ONVIF protocols. OpenCV connects to these the same way as a webcam, using the camera's stream URL instead of a device index.

```
RTSP_STREAM.PY

import cv2

rtsp_url = "rtsp://user:pass@192.168.1.64:554/stream1"
cap = cv2.VideoCapture(rtsp_url)

if not cap.isOpened():
    print("Failed to connect to stream")
else:
    ret, frame = cap.read()
    print("Frame shape:", frame.shape if ret else None)

cap.release()
```

TRY IT YOURSELF

Write a connection function that retries an RTSP stream up to 3 times with a short delay if the first attempt fails.

2.13 Model Optimization: Quantization & ONNX Export

Deployment often requires converting models to efficient formats like ONNX or TensorRT, and reducing precision (quantization) to run faster on edge devices and GPUs.

```
EXPORT_ONNX.PY

from ultralytics import YOLO

model = YOLO("yolov8n.pt")
model.export(format="onnx", imgsz=640, half=True)
# half=True enables FP16 quantization for faster inference
```

TRY IT YOURSELF

Export a trained YOLO model to ONNX format and compare the file size against the original .pt file.

2.14 Building a Real-Time Inference Pipeline

A production pipeline reads frames continuously, runs inference, post-processes results (like NMS or trigger-line logic), and outputs annotated video or structured events — balancing accuracy against frame-rate.

```
PIPELINE.PY

import cv2
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
cap = cv2.VideoCapture("input.mp4")

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    results = model(frame, verbose=False)
    annotated = results[0].plot()
    cv2.imshow("Pipeline", annotated)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
```

TRY IT YOURSELF

Extend the pipeline to log a timestamp to a list every time a detection's confidence exceeds 0.8.

2.15 Deploying CV Models with FastAPI

Wrapping a model in a REST API lets other services submit images and receive predictions, a common pattern for integrating CV into larger systems.

```
APP.PY

from fastapi import FastAPI, UploadFile
from ultralytics import YOLO
import cv2
import numpy as np

app = FastAPI()
model = YOLO("yolov8n.pt")

@app.post("/predict")
async def predict(file: UploadFile):
    data = await file.read()
    img = cv2.imdecode(np.frombuffer(data, np.uint8), cv2.IMREAD_COLOR)
    results = model(img, verbose=False)
    boxes = results[0].boxes.xyxy.tolist()
    return {"detections": boxes}
```

TRY IT YOURSELF

Add a confidence query parameter to the /predict endpoint that filters out low-confidence detections before returning.

Appendix A — Quick Reference

Handy syntax cheat sheets for both skill levels

Beginner Cheat Sheet (OpenCV Basics)

TASK	FUNCTION	EXAMPLE
Load image	<code>cv2.imread()</code>	<code>cv2.imread("a.jpg")</code>
Save image	<code>cv2.imwrite()</code>	<code>cv2.imwrite("out.jpg", img)</code>
Resize	<code>cv2.resize()</code>	<code>cv2.resize(img, (w,h))</code>
Grayscale	<code>cv2.cvtColor()</code>	<code>COLOR_BGR2GRAY</code>
Blur	<code>cv2.GaussianBlur()</code>	<code>(img, (5,5), 0)</code>
Edges	<code>cv2.Canny()</code>	<code>Canny(img, 50, 150)</code>
Draw box	<code>cv2.rectangle()</code>	<code>(img, pt1, pt2, color, 2)</code>
Video capture	<code>cv2.VideoCapture()</code>	<code>VideoCapture(0)</code>

Intermediate Cheat Sheet (Deep Learning CV)

CONCEPT	LIBRARY / API	USE CASE
Object detection	<code>YOLO(...)</code>	Real-time multi-object detection
NMS	<code>cv2.dnn.NMSBoxes()</code>	Remove duplicate overlapping boxes
Tracking	<code>model.track()</code>	Assign persistent IDs across frames
Segmentation	<code>YOLO(seg)</code>	Pixel-level object masks
Pose estimation	<code>YOLO(pose)</code>	Keypoint / joint detection
OCR	<code>easyocr.Reader()</code>	Extract text from images
Export	<code>model.export()</code>	Convert to ONNX / TensorRT
Serve model	<code>FastAPI</code>	Expose inference as a REST API

PRO TIP

Always check a model's license and dataset provenance before production use — and validate accuracy on your own footage, since benchmark scores rarely transfer perfectly to a new camera angle, lighting condition, or lens.

You've Completed the Guide

Consistent, small practice beats occasional long sessions. Revisit the tasks in this guide, then apply each concept to a real dataset or camera feed of your own.

- ☐ Rebuild every code example from memory, without looking
- ☐ Complete all 27 "Try it yourself" tasks in a code editor
- ☐ Train a small custom detector on 2–3 classes end to end
- ☐ Benchmark inference speed (FPS) on CPU vs GPU
- ☐ Read model cards and papers behind the architectures you use
- ☐ Deploy one model behind a simple API and test it with real images

COMPUTER VISION PRACTICAL GUIDE · BEGINNER TO INTERMEDIATE